Decision-Model Policy Selection for Network Digital Twin Synchronization

Authors

Gabriele Bandino <bandino@kth.se>
Jago Revrenna <revrenna@kth.se>

Abstract

A network digital twin (NDT) relies on observations that may become stale while the network changes. More frequent polling can improve the representation but increases monitoring traffic. We will test whether a decision model can choose between fixed-interval and change-aware polling as conditions change. The model will only select a policy. It will not choose individual links or predict unobserved network state. In a trace-driven simulation we will compare four controllers: each polling policy used throughout, a deterministic rule switching between them, and one decision-model selector. The main evidence will be twin-state error and application-level monitoring traffic across stable, bursty, alternating, and shifted demand with two measurement-delivery delays. We will also measure decision latency, fallbacks, and the selector's extra resource use. Jev and an open-weight model are candidates; we will select one on feasibility grounds in a pilot and freeze it before held-out runs. The result will show when model-based switching changes the trade-off between accuracy and traffic, including cases where a simpler controller is sufficient.

1 Introduction

The project concerns one part of synchronization: deciding which polling policy should control the next period of observations. This is a narrower claim than improving an entire operational digital twin, but it gives us a decision that can be implemented and tested within the course.

1.1 Background and Rationale

The Network Digital Twin reference architecture describes a digital representation maintained using network observations and models [1]. Our experimental twin will hold the latest received utilization value for each link in a small fixed graph. A value can become inaccurate between observations. Polling frequently limits how long the twin holds old measurements, while producing more monitoring messages.

A fixed-interval policy offers predictable overhead. A change-aware policy instead changes its interval after observing variation; this may save traffic in quiet periods, but its threshold and interval bounds matter. We ask whether a decision model can select between these two policies using the same information available to a simple switching rule. The question is *which policy to use at a review point*, not which link to query. At each poll, the chosen policy measures the same fixed set of links. Push updates and learned prediction are outside the study.

1.2 Theory and Related Work

OpenNetMon adapts its polling rate as observed flow rates change [2]. It motivates our change-aware baseline, which will be a small project implementation rather than a reproduction of its per-flow system. Wieme, Baert, and Hoebeke examine how monitoring frequency and monitoring traffic affect a Bluetooth Mesh digital twin network [3]. Their setting differs from ours, but the paper supports judging accuracy together with communication cost.

Age of information (AoI) measures the time since the freshest received update was generated [4]. It is not the same as state error: an old value can still describe an unchanged link. Both age and past observed changes are useful inputs to a selector, but neither reveals a current unmeasured event. We will prevent the selector from seeing the true trace or any future measurement.

Jev's Choice interface selects from specified options and returns associated probabilities [5]. This makes Jev a candidate for choosing between *fixed-interval* and *change-aware* polling. An open-weight decision model such as Laya is another candidate [6]. The same surrounding code would calculate summaries, validate the choice, and execute the selected policy in either case. TypeSafe describes request-based customization for Jev [7] and warns against relying on its numerical precision [8]; these claims do not automatically apply to other models. Neither the cited studies nor the model documentation establish that any decision model improves network-twin synchronization. That is what our experiment will test, using one model selected and fixed before evaluation.

1.3 Research Problem, Aim, and Research Questions

The problem is choosing how often to refresh a twin when current network state cannot be observed without polling. We aim to determine whether one decision-model controller improves the trade-off between accuracy and communication by selecting between two conventional polling policies, compared with using either policy continuously or switching by a deterministic rule.

Main question. Under which tested traffic conditions, if any, does decision-model selection between fixed-interval and change-aware polling reduce twin-state error at comparable monitoring traffic, relative to the three simpler controllers?

Supporting question. How do measurement-delivery delay, selector response time, fallbacks, and extra communication or computation affect the practical benefit?

We expect that switching may matter most when quiet and rapidly changing periods alternate, but the change-aware policy or simple rule may perform just as well. These are expectations to test. Which model we use is an implementation choice made in the pilot; it is not an experimental factor, and we make no claim that one candidate outperforms another.

Expected contribution. We will produce a four-controller, scenario-by-scenario comparison: plots of error against traffic, paired differences at overlapping monitoring-traffic levels, and a separate account of selector latency and resource use. This will identify conditions in which model-based switching changes the trade-off and where simpler alternatives suffice.

2 Research Method

We will use a quantitative trace-driven simulation. This provides known ground truth and permits paired comparisons on exactly the same changing network conditions. The design, pilot, analysis, and checks below specify how each question will be answered.

2.1 Research Design and Methodological Rationale

The simulation will generate link-utilization traces from synthetic demands routed along fixed paths. A controller sees only measurements received by the twin. The evaluator sees the true trace to calculate error. The twin stores each link's last received utilization until a newer measurement arrives. Every poll samples the same set of links, so policy choice changes *when* the twin observes them, not what it observes.

Four controllers will be compared: fixed-interval polling, change-aware polling, threshold-rule switching, and switching by one selected decision model. The latter two choose a policy at preset review times. Both selectors receive the same types of summary: timestamps and values already received, observed changes, ages of current measurements, and recent poll counts. Their realized histories

may differ because earlier choices affect what they subsequently observe. Neither can inspect the unmeasured current state. The comparison against both static policies tests whether switching is useful at all; the rule baseline tests whether the model adds value beyond a cheap switch.

Table 1: Components of the experimental comparison.

Category	Choices or recorded attributes
Controllers	Fixed-interval; change-aware; threshold-rule switching; decision-model switching. The same two polling policies are available to both switchers.
Network conditions	One small fixed topology; stable traffic, temporary bursts, alternating regimes, and persistent load shifts; low or elevated measurement-delivery delay.
Measures	Primary: mean absolute twin-state error and application-level monitoring bytes. Supporting: poll count, detection delay, decision latency, fallback frequency, and model overhead (API bytes/charges if hosted; runtime and hardware if local).

A physical testbed would capture transport and equipment effects better. We instead isolate the polling decision and can replay identical traces within the available time. Conclusions will concern this synchronization subsystem, not all digital twins or all networks.

2.2 Data, Materials, and Experimental Conditions

The provisional environment is a fixed graph with around eight monitored links and shared paths. Demand changes will create four traffic regimes: nearly stable load, short bursts, alternating quiet and changing periods, and a persistent shift. We will replay each under low and elevated measurement-delivery delay. The pilot will determine the numerical amplitudes, duration, delays, and a feasible number of independent seeds. For planning, a five-minute trace, three candidate fixed intervals spanning short to long polling, and about 12–20 held-out seeds per condition are starting estimates, not values chosen to favour the model-based controller. We will record the final choices before main runs.

Python will implement the trace generator, twin update logic, policies, and log analysis. During the pilot, we will assess Jev's Choice API [5] and an open-weight candidate such as Laya [6] for the same two-option decision. We will record whether each is accessible, supports a reliably parseable choice, meets a prespecified decision deadline, and can be run with available resources and no personal expenditure. This is a feasibility screen, not a comparative performance study. We will choose one suitable model, record why, and freeze its identifier or weights, request format, decoding settings, and input summary before held-out experiments. If neither is feasible, we will ask the course mentor to revise the plan before main runs. No model training or live network is required. Only synthetic summaries will be used; an external service will receive no operational or personal data.

2.3 Work Procedure and Data Collection

We will first implement the trace generator and twin updater, then the two polling policies. The fixed policy polls at one selected interval. The change-aware policy shortens its interval after sufficiently large observed changes and lengthens it after quiet periods, within preset bounds. The thresholds and bounds will be chosen using pilot traces and frozen before evaluation. Both static and switching controllers will use the same frozen policy implementations and settings.

Next, we will implement two selectors. A deterministic rule will use preset thresholds on observed change and age. The selected model will receive a compact, structured summary of the same information and choose *fixed* or *change-aware*; it will not produce a new interval or a numeric prediction.

The exact request format, review frequency, and timeout will be finalized in the pilot. No selector prompt or threshold will be revised using held-out results. The selected policy will take effect when its decision arrives. Until then, the previous policy stays active. At a switch, the existing measurement history is retained, and the next poll is scheduled from the last poll time without an extra poll.

A poll will sample the true utilization at dispatch; its result reaches the twin after the scenario's delivery delay. The twin will reject a measurement older than its latest accepted one. If the selected model's answer is invalid or misses a preset deadline, the controller retains the previous policy and records a fallback. Only one model request will be outstanding; a later review will not create a backlog. We will test that a pending answer cannot use measurements received after its request was formed. This prevents look-ahead and makes decision latency an operational part of the experiment.

The pilot will run short traces to validate the code and determine final scenario values, interval and threshold settings, review period, timeout, and repetition count. The main experiment will use new seeds shared across all four controllers. We will record simulator and model versions, seeds, traffic and policy configurations, true and twin states, poll dispatch and delivery times, review inputs, selected policies, decision times, model failures, and message sizes. The exact model identifier or local weights, configuration, and usage will be logged; credentials will not be saved. We will inspect a small sample of run logs manually before collecting the full set.

2.4 Data Analysis and Quality Assurance

For the main question, mean absolute state error will average the absolute difference between true and stored link utilization over all monitored links and evaluation times after a common warm-up. We will report it in utilization percentage points. Monitoring cost will be the number and application-level bytes of polling requests and replies; the warm-up's messages will remain in cost totals. A lower error obtained only through more polling will not be called a better selection decision.

For each traffic and delivery-delay condition, we will plot error against monitoring bytes across settings frozen in the pilot. We will compare controllers at overlapping traffic ranges; if ranges do not overlap, we will report the trade-off without ranking them as if costs were equal. Decision latency and failures will be reported separately. For a hosted model we will also record request and reply bytes and any charges; for a local model we will record runtime and hardware used. We will distinguish selector overhead from monitoring traffic instead of treating local computation and network bytes as equivalent. We will not label payload byte counts as wire-level traffic, energy, or carbon use.

We will pair runs by workload seed and present the direction and size of differences with uncertainty intervals based on complete paired runs. Time samples from one run will not be treated as independent replicates. We will report all tested regimes, including null or unfavourable outcomes. Supporting analyses will examine how long scheduled changes take to appear in the twin and how often the model selector falls back. We will distinguish model variation on an identical trace from variation among independently generated traffic traces.

Before interpreting results, we will test constant-state input, known hand-worked error calculations, policy switching without an extra poll, delayed and out-of-order replies, timeouts, and the absence of future-state leakage. Both authors will review each other's component and inspect paired sample runs. The pilot and main-run seed sets will be separated; configurations and analysis choices will be frozen and archived before the main evaluation. A failure to detect an unobserved, unpredictable change will be treated as a limitation shared by all observation-based policies, not as evidence unique to the selected model. Limits include synthetic traces, one topology, two predefined policies, and use of only one decision model; results will not generalize to all models.

2.5 Use of AI Tools

AI assistance has been used in preparing this plan, which both authors will review and revise. During implementation, we may use AI tools for language feedback, coding support, and literature discovery. We will check cited claims against their sources, inspect generated code, and validate results independently of generated explanations. Material assistance will be recorded in accordance with course requirements. The selected model's role as a tested controller is distinct from writing or coding assistance: its inputs, decisions, and failures are research data, not ground truth.

3 Project Planning and Feasibility

The plan separates a feasibility gate from the main runs. Dates below are working targets from 7 October onward; official submission and seminar dates will be checked in Canvas and with the course mentor. A pilot problem will be resolved before freezing the main protocol, not hidden by changing settings after observing test results.

3.1 Work Tasks and Milestones

Each stage has a deliverable that allows us and the mentor to judge whether the next stage is realistic.

Table 2: Working milestones and completion evidence.

Target window	Deliverable or decision
7–9 Oct	Confirm scope with the course mentor; screen candidate models for access, latency, stable output, hardware needs, and zero personal cost. Record a go/no-go decision.
10–16 Oct	Build generator, twin updater, fixed and change-aware policies, and threshold switch; pass basic invariant tests.
17–23 Oct	Integrate one selected model; complete the pilot; freeze its version, inputs, scenarios, policy settings, review period, timeout, seeds, and analysis rules.
24 Oct–7 Nov	Run held-out paired experiments, inspect logs, and repeat a predefined subset of model-selector runs.
8–20 Nov	Analyze results and uncertainty; review limitations; prepare the first report draft and figures.
Late Nov–Dec	Revise after mentor feedback and prepare the report and presentation according to the confirmed course schedule.

3.2 Allocation of Responsibilities

Our proposed division is for Jago to lead the simulator, twin updater, and conventional policies, and for Gabriele to lead the candidate-model feasibility check, selected-model interface, and logging. Each will review the other's code and a sample of its outputs. Both will agree the pilot freeze, read and verify sources, run the analysis, and write the interpretation. We will keep code, configuration, and decisions in a shared repository and review progress at least weekly. This allocation is a working agreement to confirm together; both authors must be able to explain the whole experiment.

3.3 Implementational Risks and Mitigation Strategies

Model access, runtime, and cost. The pilot is a gate. We will choose one model that can make the two-option decision within a practical deadline on available infrastructure without personal expenditure. Hosted and local options have different constraints, so we will record the choice and its reasons. If neither candidate is feasible, we will bring the measured constraints and options to the course mentor

before main runs rather than silently changing the experiment. Timeouts and fallback rates will remain visible.

Weak or misleading comparison. Jev's own documentation notes limits in numerical precision [8]; no candidate is assumed to be good at this task. The deterministic rule is therefore a necessary baseline. We will freeze the selected model's request and the baselines on development traces, use held-out paired seeds, and compare error only at comparable monitoring traffic. A negative result remains an answer to the research question.

Simulation and reproducibility. One graph and synthetic demand patterns limit generalization. We will document the generator, seeds, code version, policy settings, selected model and its configuration, and analysis script. Recorded selector inputs and outputs will support replay of the analysis even if a hosted service later changes. We will not interpret a missing answer as a successful selection. If the pilot reveals a design fault, we will document the repair and refreeze the protocol before held-out evaluation.

No human participants, private operational traces, or live network controls are needed. We will use only synthetic inputs for the model, including if it is hosted. Resource use will be bounded during the pilot; fewer application messages will not be presented as measured energy savings.

References

[1] C. Zhou, H. Yang, X. Duan, D. Lopez, A. Pastor, Q. Wu, M. Boucadair, and C. Jacquenet, "Network Digital Twin (NDT): Concepts and Reference Architecture," IRTF Network Management Research Group, Internet-Draft draft-irtf-nmrg-network-digital-twin-arch-13, Jul. 2026. Work in progress, intended status: Informational. Accessed: 5 Nov. 2026. [Online]. Available: `https://datatracker.ietf.org/doc/html/draft-irtf-nmrg-network-digital-twin-arch-13`

[2] N. L. M. van Adrichem, C. Doerr, and F. A. Kuipers, "OpenNetMon: Network monitoring in OpenFlow Software-Defined Networks," in *2014 IEEE Network Operations and Management Symposium (NOMS)*, 2014, pp. 1–8. doi: 10.1109/NOMS.2014.6838228.

[3] J. Wieme, M. Baert, and J. Hoebeke, "The Role of Monitoring Frequency in Optimizing Digital Twin Networks," in *NOMS 2025–2025 IEEE Network Operations and Management Symposium*, 2025, pp. 1–7. doi: 10.1109/NOMS57970.2025.11073664.

[4] R. D. Yates, Y. Sun, D. R. Brown III, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of Information: An Introduction and Survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, May 2021. doi: 10.1109/JSAC.2021.3065072.

[5] TypeSafe AI, "Choice," product documentation. Accessed: 5 Nov. 2026. [Online]. Available: `https://docs.typesafe.ai/primitives/choice`

[6] Convai Innovations, "Laya," model card. Accessed: 5 Nov. 2026. [Online]. Available: `https://huggingface.co/convaiinnovations/laya`

[7] TypeSafe AI, "Models," product documentation. Accessed: 5 Nov. 2026. [Online]. Available: `https://docs.typesafe.ai/models`

[8] TypeSafe AI, "Jev 1.13 jaggedness," product documentation, reviewed 2 Oct. 2026. Accessed: 5 Nov. 2026. [Online]. Available: `https://docs.typesafe.ai/model-jaggedness/jev-1.13`